

Numerical Methods for the Group LASSO Problem

Convex Optimization Programming Assignment

Yuhan Ye

Abstract

This report studies the group LASSO benchmark from Homework 5 of the course *Convex Optimization*. The methods considered include direct CVX formulations solved by MOSEK and Gurobi, equivalent conic reformulations implemented directly for MOSEK and Gurobi, a subgradient method for the primal problem, a gradient method for a smoothed primal model, the proximal-gradient method, the fast proximal-gradient method (FISTA), a dual augmented Lagrangian method (ALM), a dual alternating direction method of multipliers (ADMM), and a linearized ADMM method for the primal splitting formulation. The report presents the problem formulation, derives the main update rules for these methods, and compares their numerical performance on the standard benchmark instance. The results show that all tested methods achieve essentially the same objective value and exact recovery accuracy, while proximal and operator-splitting methods are substantially more efficient than subgradient-type methods. Among the tested solvers, the dual ADMM method achieves the best runtime on this instance.

1 Problem Statement and Benchmark Setting

1.1 Problem statement

The programming assignment considers the group LASSO problem

$$\min_{x \in \mathbb{R}^{n \times l}} F(x) := \frac{1}{2} \|Ax - b\|_F^2 + \mu \|x\|_{1,2}, \quad (1)$$

where $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^{m \times l}$, $\mu > 0$, and

$$\|x\|_{1,2} = \sum_{i=1}^n \|x_{i,:}\|_2.$$

Here $x_{i,:}$ denotes the i th row of x . The regularizer is row-sparse and promotes solutions whose nonzero entries are concentrated in a small number of rows.

The homework consists of the following tasks:

1. Solve (1) in CVX using different solvers, including MOSEK and Gurobi.
2. Reformulate (1) into an equivalent model that can be handled directly by MOSEK and Gurobi, and implement the corresponding code.
3. Implement several iterative methods. The methods treated below are Problems 3(a), 3(b), 3(d), 3(e), 3(f), 3(g), and 3(h).

For convenience, write

$$g(x) = \frac{1}{2} \|Ax - b\|_F^2, \quad h(x) = \|x\|_{1,2}, \quad F(x) = g(x) + \mu h(x).$$

1.2 Benchmark instance and evaluation metrics

The standard test instance follows the homework script exactly. The random seed is fixed as

$$\text{seed} = 97006855,$$

and the data are generated by

$$n = 512, \quad m = 256, \quad l = 2, \quad k = \text{round}(0.1n), \quad \mu = 10^{-2}.$$

A matrix $A \in \mathbb{R}^{m \times n}$ is drawn from the standard Gaussian distribution, k rows of the ground-truth signal $u \in \mathbb{R}^{n \times l}$ are selected uniformly at random and filled with Gaussian entries, and the observation is generated by

$$b = Au.$$

This produces a noiseless sparse-recovery problem with a row-sparse exact solution.

To compare accuracy and efficiency, the following quantities are recorded:

$$\text{Objective value: } F(x) = \frac{1}{2} \|Ax - b\|_F^2 + \mu \|x\|_{1,2},$$

$$\text{Exact recovery error: } \text{Err}_{\text{exact}}(x) = \frac{\|x - u\|_F}{1 + \|u\|_F},$$

$$\text{Sparsity ratio: } \text{Sparsity}(x) = \frac{\#\{(i, j) : |x_{ij}| > 10^{-6} \max |x|\}}{nl},$$

where $\max |x|$ is the maximum absolute entry of x . CPU time and iteration counts are also reported whenever they are available.

2 Summary of Numerical Results

Table 1 summarizes the numerical results. The labels follow the required file names in the homework specification. In particular, the routine name `gl_SGD_primal` corresponds to the deterministic subgradient method in Problem 3(a), which is the naming convention used in the course materials.

Table 1: Comparison of all tested solvers on the standard benchmark.

Solver	Objective	Sparsity	Exact error	Time (s)	Iterations
1 <code>gl_cvx_mosek</code>	5.38327e-01	0.115	4.22e-05	11.85	—
1 <code>gl_cvx_gurobi</code>	5.38327e-01	0.114	4.22e-05	12.22	—
2 <code>gl_mosek</code>	5.38315e-01	0.119	8.27e-05	19.57	—
2 <code>gl_gurobi</code>	5.38327e-01	0.112	4.16e-05	19.77	—
3(a) <code>gl_SGD_primal</code>	5.38327e-01	0.100	4.06e-05	7.39	8950
3(b) <code>gl_GD_primal</code>	5.38327e-01	0.100	4.06e-05	7.43	11475
3(d) <code>gl_ProxGD_primal</code>	5.38327e-01	0.100	4.06e-05	0.94	1476
3(e) <code>gl_FProxGD_primal</code>	5.38327e-01	0.100	3.99e-05	0.94	1423
3(f) <code>gl_ALM_dual</code>	5.38327e-01	0.114	4.20e-05	0.69	1207
3(g) <code>gl_ADMM_dual</code>	5.38326e-01	0.113	4.20e-05	0.19	345
3(h) <code>gl_ADMM_primal</code>	5.38348e-01	0.104	5.04e-05	0.46	2929

All methods reach essentially the same objective value, around 5.383×10^{-1} , and all exact recovery errors are on the order of 10^{-5} . The main difference lies in computational efficiency. The

subgradient and smoothed-gradient methods are clearly slower, whereas proximal-gradient, dual ALM, and especially dual ADMM are much more efficient. On this benchmark, the best wall-clock time is achieved by `gl_ADMM_dual`, while `gl_FProxGD_primal` gives the smallest exact recovery error among the reported methods.

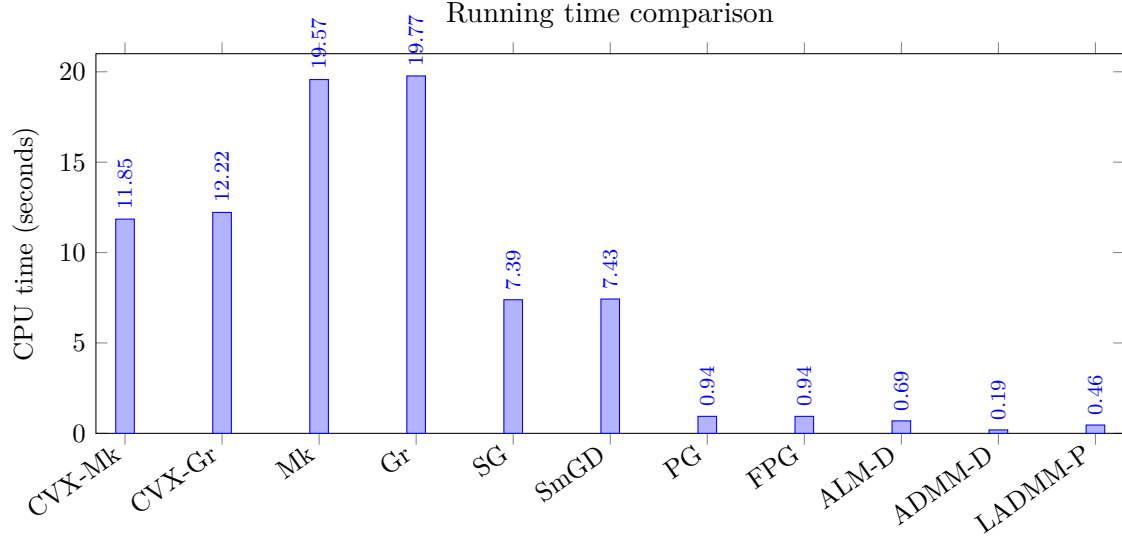


Figure 1: CPU-time comparison of all methods. The dual ADMM solver is the fastest on the test instance.

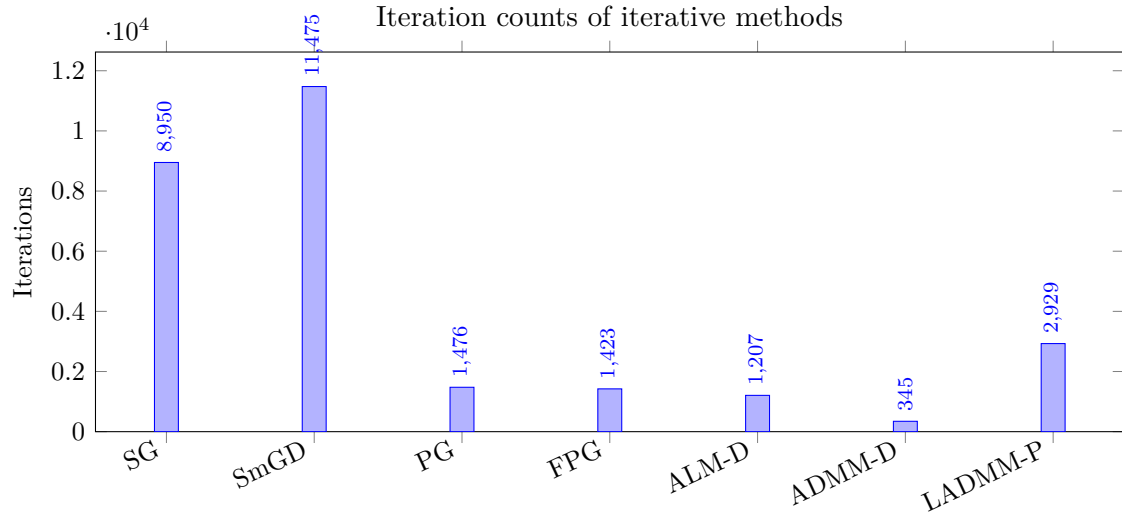


Figure 2: Iteration counts for iterative methods. Proximal and splitting methods require far fewer iterations than subgradient-type methods.

3 Problems 1 and 2: CVX and Direct Conic Reformulation

3.1 Problem 1: CVX formulation

The model in (1) can be expressed directly in CVX and then solved by MOSEK or Gurobi. These runs provide reliable references for the later algorithmic comparisons. As shown in Table 1, both CVX-based solvers recover nearly identical solutions, with objective values around 5.38327×10^{-1} and exact errors around 4.22×10^{-5} .

3.2 Problem 2: direct reformulation for MOSEK and Gurobi

The original matrix form is not convenient for the low-level solver interfaces, so it is useful to rewrite the problem as a vector conic program.

Stack the rows of x into a vector $\tilde{x} \in \mathbb{R}^{nl}$, and stack the rows of b into $\tilde{b} \in \mathbb{R}^{ml}$. Let $\tilde{A}$ be the corresponding lifted matrix such that

$$\tilde{A}\tilde{x} - \tilde{b} = \text{vec}_r(Ax - b).$$

This row-wise stacking keeps each group contiguous:

$$\mathcal{G}_i = \{(i-1)l + 1, \dots, il\}, \quad \|x\|_{1,2} = \sum_{i=1}^n \|\tilde{x}_{\mathcal{G}_i}\|_2.$$

Hence (1) is equivalent to

$$\min_{\tilde{x} \in \mathbb{R}^{nl}} \frac{1}{2} \|\tilde{A}\tilde{x} - \tilde{b}\|_2^2 + \mu \sum_{i=1}^n \|\tilde{x}_{\mathcal{G}_i}\|_2. \quad (2)$$

Introduce residual and epigraph variables $r \in \mathbb{R}^{ml}$ and $s \in \mathbb{R}^n$. Then (2) becomes

$$\begin{aligned} \min_{\tilde{x}, r, s} \quad & \frac{1}{2} \|r\|_2^2 + \mu \sum_{i=1}^n s_i \\ \text{s.t.} \quad & \tilde{A}\tilde{x} - \tilde{b} = r, \\ & \|\tilde{x}_{\mathcal{G}_i}\|_2 \leq s_i, \quad i = 1, \dots, n. \end{aligned} \quad (3)$$

This is already a convex quadratic-conic model. To obtain a linear objective for a conic solver, introduce one more scalar $v \geq 0$ such that

$$\|r\|_2^2 \leq 2v.$$

The model then becomes

$$\begin{aligned} \min_{\tilde{x}, r, s, v} \quad & v + \mu \sum_{i=1}^n s_i \\ \text{s.t.} \quad & \tilde{A}\tilde{x} - \tilde{b} = r, \\ & \|\tilde{x}_{\mathcal{G}_i}\|_2 \leq s_i, \quad i = 1, \dots, n, \\ & \|r\|_2^2 \leq 2v, \quad v \geq 0. \end{aligned} \quad (4)$$

The last inequality is a rotated second-order cone constraint, so (4) is suitable for MOSEK.

For Gurobi, the same model can be written as a QCQP by squaring the norm constraints and adding the required nonnegativity conditions:

$$\begin{aligned}
& \min_{\tilde{x}, r, s, v} \quad v + \mu \sum_{i=1}^n s_i \\
& \text{s.t.} \quad \tilde{A}\tilde{x} - \tilde{b} = r, \\
& \quad \|\tilde{x}_{\mathcal{G}_i}\|_2^2 \leq s_i^2, \quad s_i \geq 0, \quad i = 1, \dots, n, \\
& \quad \|r\|_2^2 \leq 2v, \quad v \geq 0.
\end{aligned} \tag{5}$$

The numerical results in Table 1 show that the direct MOSEK and Gurobi models are as accurate as the CVX baselines, although they are not faster in absolute CPU time on this benchmark.

4 Problem 3(a): Subgradient Method for the Primal Problem

4.1 Subgradient derivation

The smooth part $g(x) = \frac{1}{2} \|Ax - b\|_F^2$ is differentiable everywhere. Expanding the Frobenius norm gives

$$g(x) = \frac{1}{2} \text{tr}((Ax - b)^\top (Ax - b)) = \frac{1}{2} \text{tr}(x^\top A^\top Ax) - \text{tr}(b^\top Ax) + \frac{1}{2} \text{tr}(b^\top b),$$

so its gradient is

$$\nabla g(x) = A^\top (Ax - b). \tag{6}$$

For the nonsmooth term,

$$h(x) = \sum_{i=1}^n \|x_{i,:}\|_2,$$

the subdifferential is blockwise separable. For every row i ,

$$\partial \|x_{i,:}\|_2 = \begin{cases} \left\{ \frac{x_{i,:}}{\|x_{i,:}\|_2} \right\}, & x_{i,:} \neq 0, \\ \left\{ g \in \mathbb{R}^{1 \times l} : \|g\|_2 \leq 1 \right\}, & x_{i,:} = 0. \end{cases}$$

Therefore, a subgradient $G \in \partial h(x)$ may be chosen row by row. In the implementation, the zero vector is selected whenever $x_{i,:} = 0$. The primal subgradient iteration is

$$x^{k+1} = x^k - \tau_k (\nabla g(x^k) + \mu G^k), \quad G^k \in \partial h(x^k). \tag{7}$$

4.2 Implementation details

Three practical devices are used to stabilize and accelerate the method:

1. **Continuation in μ .** The run starts from $\mu^{(0)} = 2^8 \mu$, performs at most 1000 subgradient steps, and then halves μ stage by stage until the target value is reached. This warm-start strategy substantially reduces the difficulty of the early stages.
2. **Adaptive step sizes.** A larger step size 10^{-3} is used in the warm-start stages, while a smaller step size 10^{-5} is used once the target regularization parameter is reached.
3. **Thresholding small entries.** To better exploit sparsity, entries with magnitude below 10^{-3} are rounded to zero in the warm-start stages, and the threshold is tightened to 10^{-5} at the final stage.

4.3 Discussion

The subgradient method is reliable but slow. It reaches the correct objective value and a small exact recovery error, but it requires 8950 iterations and 7.39 seconds on the benchmark. This is much slower than the proximal and splitting methods considered below.

5 Problem 3(b): Gradient Method for the Smoothed Primal Problem

5.1 Moreau smoothing of the row norm

A differentiable approximation can be obtained by applying the Moreau envelope to each row norm. For a vector $u \in \mathbb{R}^l$, define

$$M_\alpha(u) = \inf_{z \in \mathbb{R}^l} \left\{ \|z\|_2 + \frac{1}{2\alpha} \|z - u\|_2^2 \right\}, \quad \alpha > 0.$$

The minimizer is the proximal operator of the Euclidean norm:

$$\text{prox}_{\alpha\|\cdot\|_2}(u) = \begin{cases} \left(1 - \frac{\alpha}{\|u\|_2}\right)u, & \|u\|_2 \geq \alpha, \\ 0, & \|u\|_2 < \alpha. \end{cases}$$

Substituting this minimizer into the definition yields the explicit envelope formula

$$M_\alpha(u) = \begin{cases} \|u\|_2 - \frac{\alpha}{2}, & \|u\|_2 \geq \alpha, \\ \frac{1}{2\alpha} \|u\|_2^2, & \|u\|_2 < \alpha. \end{cases} \quad (8)$$

Its gradient is

$$\nabla M_\alpha(u) = \begin{cases} \frac{u}{\|u\|_2}, & \|u\|_2 \geq \alpha, \\ \frac{u}{\alpha}, & \|u\|_2 < \alpha. \end{cases} \quad (9)$$

The smoothed model is therefore

$$\min_x g(x) + \mu \sum_{i=1}^n M_\alpha(x_{i,:}),$$

which is differentiable for every fixed $\alpha > 0$.

5.2 Gradient iteration and parameter schedule

Using (6) and (9), the gradient step is

$$x^{k+1} = x^k - \tau_k \left(A^\top (Ax^k - b) + \mu \nabla M_{\alpha_k}(x^k) \right),$$

where $\nabla M_{\alpha_k}(x^k)$ is applied row by row. The same continuation and thresholding strategies as in the subgradient method are used. The smoothing parameter starts from $\alpha_0 = 10^{-2}$ and is reduced stage by stage. Once α_k becomes sufficiently small, the final refinement closely matches the original nonsmooth model.

5.3 Discussion

Numerically, the smoothed-gradient method behaves similarly to the plain subgradient method. Its final objective value and exact recovery error are almost identical, but the running time is not improved in a meaningful way. This reflects the fact that smoothing removes the kink in the penalty but does not exploit the proximal structure of the row-sparsity term as directly as the next methods do.

6 Problems 3(d) and 3(e): Proximal Gradient and FISTA

6.1 Proximal gradient method

The proximal-gradient method is particularly suitable for (1) because the smooth and nonsmooth parts are naturally separated. For a step size $\tau_k > 0$, the proximal-gradient update solves

$$x^{k+1} = \arg \min_x \left\{ \langle \nabla g(x^k), x - x^k \rangle + \frac{1}{2\tau_k} \|x - x^k\|_F^2 + \mu \|x\|_{1,2} \right\},$$

which is equivalent to

$$x^{k+1} = \text{prox}_{\tau_k \mu \|\cdot\|_{1,2}} \left(x^k - \tau_k A^\top (Ax^k - b) \right). \quad (10)$$

The proximal operator is rowwise block soft-thresholding. For a matrix $Y \in \mathbb{R}^{n \times l}$,

$$(\text{prox}_{\tau \mu \|\cdot\|_{1,2}}(Y))_{i,:} = \left(1 - \frac{\tau \mu}{\|Y_{i,:}\|_2} \right)_+ Y_{i,:}, \quad (a)_+ := \max\{a, 0\}. \quad (11)$$

This closed form is the main reason for the large speedup over the subgradient methods.

The implementation again uses continuation in μ , but with a more aggressive schedule: it starts from $\mu^{(0)} = 2^{16} \mu$, divides by 4 at each stage, and performs at most 100 proximal-gradient iterations per stage. The step size is 10^{-3} in the warm-start stages and 10^{-4} at the final stage. The same thresholding strategy is retained.

6.2 Fast proximal gradient method (FISTA)

FISTA adds Nesterov-type extrapolation to proximal gradient. A standard form is

$$\begin{aligned} t_1 &= 1, \\ y^k &= x^k + \frac{t_k - 1}{t_{k+1}} (x^k - x^{k-1}), \end{aligned} \quad (12)$$

$$x^{k+1} = \text{prox}_{\tau_k \mu \|\cdot\|_{1,2}} \left(y^k - \tau_k A^\top (Ay^k - b) \right), \quad (13)$$

$$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}.$$

The only difference from (10) is that the gradient is evaluated at the extrapolated point y^k .

6.3 Discussion

The proximal-gradient family is dramatically faster than the subgradient family. On the benchmark, `gl_ProxGD_primal` reduces the running time from about 7.4 seconds to 0.94 seconds while

maintaining essentially the same objective value and exact recovery error. FISTA further reduces the iteration count from 1476 to 1423. The improvement is visible but modest, because the continuation strategy already provides good warm starts; once the target value of μ is reached, the iterate is already close to the optimum.

7 Problems 3(f) and 3(g): Dual ALM and Dual ADMM

7.1 Derivation of the dual problem

Introduce the residual variable $R = Ax - b$. Then the primal problem is equivalent to

$$\min_{x,R} \frac{1}{2} \|R\|_F^2 + \mu \|x\|_{1,2} \quad \text{s.t.} \quad Ax - b = R.$$

Let $Z \in \mathbb{R}^{m \times l}$ be the Lagrange multiplier associated with the constraint $Ax - b - R = 0$. The Lagrangian is

$$\mathcal{L}(x, R; Z) = \frac{1}{2} \|R\|_F^2 + \mu \|x\|_{1,2} + \langle Z, Ax - b - R \rangle.$$

Minimizing over R gives

$$\inf_R \left(\frac{1}{2} \|R\|_F^2 - \langle Z, R \rangle \right) = -\frac{1}{2} \|Z\|_F^2.$$

Minimizing over x gives a finite value if and only if the dual norm constraint is satisfied:

$$\|A^\top Z\|_{\infty,2} \leq \mu, \quad \|Y\|_{\infty,2} := \max_{1 \leq i \leq n} \|Y_{i,:}\|_2.$$

Therefore the dual problem is

$$\max_{Z \in \mathbb{R}^{m \times l}} -\frac{1}{2} \|Z\|_F^2 - \langle Z, b \rangle \quad \text{s.t.} \quad \|A^\top Z\|_{\infty,2} \leq \mu, \quad (14)$$

or equivalently,

$$\min_{Z \in \mathbb{R}^{m \times l}} \frac{1}{2} \|Z\|_F^2 + \langle Z, b \rangle \quad \text{s.t.} \quad \|A^\top Z\|_{\infty,2} \leq \mu. \quad (15)$$

This is the form used by the dual ALM and dual ADMM solvers.

To decouple the constraint, introduce an auxiliary variable $V \in \mathbb{R}^{n \times l}$ and the set

$$\mathcal{B}_\mu := \{V \in \mathbb{R}^{n \times l} : \|V\|_{\infty,2} \leq \mu\}.$$

Then (15) becomes

$$\min_{Z,V} \frac{1}{2} \|Z\|_F^2 + \langle Z, b \rangle + I_{\mathcal{B}_\mu}(V) \quad \text{s.t.} \quad A^\top Z - V = 0, \quad (16)$$

where $I_{\mathcal{B}_\mu}$ is the indicator function of $\mathcal{B}_\mu$.

The projection onto $\mathcal{B}_\mu$ is rowwise:

$$(\text{Proj}_{\mathcal{B}_\mu}(Y))_{i,:} = \begin{cases} \mu \frac{Y_{i,:}}{\|Y_{i,:}\|_2}, & \|Y_{i,:}\|_2 > \mu, \\ Y_{i,:}, & \|Y_{i,:}\|_2 \leq \mu. \end{cases} \quad (17)$$

7.2 Dual augmented Lagrangian method

For (16), the augmented Lagrangian with penalty parameter $\beta > 0$ is

$$\mathcal{L}_\beta(Z, V; \Lambda) = \frac{1}{2} \|Z\|_F^2 + \langle Z, b \rangle + I_{\mathcal{B}_\mu}(V) + \langle \Lambda, A^\top Z - V \rangle + \frac{\beta}{2} \|A^\top Z - V\|_F^2.$$

At each outer iteration, the ALM subproblem is minimized approximately by alternating between V and Z .

For fixed Z and Λ , the V -update is simply the projection

$$V = \text{Proj}_{\mathcal{B}_\mu} \left(A^\top Z + \frac{\Lambda}{\beta} \right). \quad (18)$$

For fixed V and Λ , differentiating with respect to Z gives

$$0 = Z + b + A\Lambda + \beta A(A^\top Z - V),$$

so the Z -update solves

$$(I_m + \beta AA^\top) Z = \beta AV - A\Lambda - b. \quad (19)$$

Finally, the multiplier is updated by

$$\Lambda^{k+1} = \Lambda^k + \beta(A^\top Z^{k+1} - V^{k+1}). \quad (20)$$

Each ALM subproblem is solved approximately by a small number of inner alternating steps using (18)–(19). A continuation strategy in μ is also used, which substantially improves efficiency on this benchmark.

Figure 3 shows the convergence behavior of the dual ALM method. The full-scale plot gives the overall evolution, while the two zoomed views highlight the rapid stabilization once the continuation parameter approaches its target value.

7.3 Dual ADMM

Applying ADMM to (16) leads to the updates

$$(I_m + \beta AA^\top) Z^{k+1} = \beta AV^k - A\Lambda^k - b, \quad (21)$$

$$V^{k+1} = \text{Proj}_{\mathcal{B}_\mu} \left(A^\top Z^{k+1} + \frac{\Lambda^k}{\beta} \right), \quad (22)$$

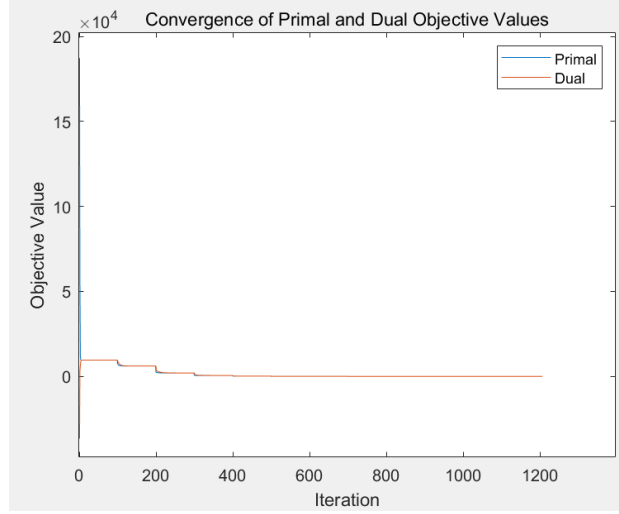
$$\Lambda^{k+1} = \Lambda^k + \beta(A^\top Z^{k+1} - V^{k+1}). \quad (23)$$

These are the same building blocks as in the ALM subproblem, but they are now used directly as the main iteration. In the reported experiment, the penalty parameter is set to $\beta = 0.1$. A short warm-start phase on μ is used only during the first 20 iterations.

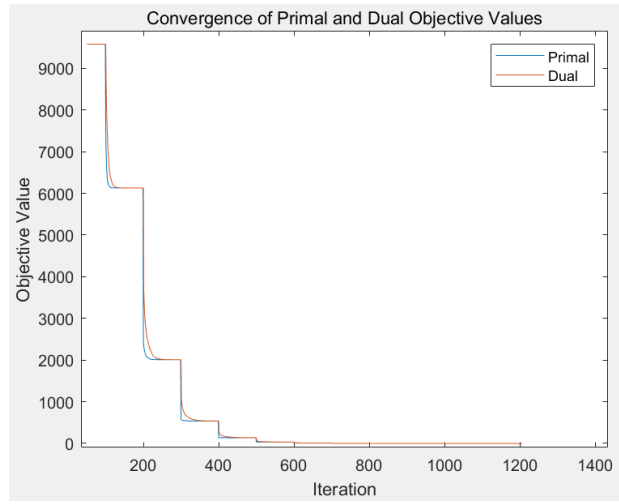
Figure 4 shows the convergence behavior of the dual ADMM method. The primal objective exhibits noticeable oscillations during the early iterations, while the dual objective stabilizes more quickly.

7.4 Discussion

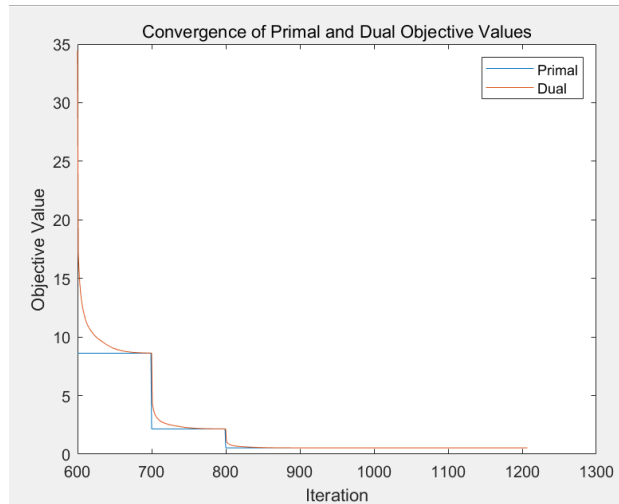
Both dual ALM and dual ADMM are highly competitive. The ALM solver already outperforms proximal gradient in running time, and the dual ADMM solver is the fastest method in the comparison: it reaches the correct objective value in only 345 iterations and 0.19 seconds. This behavior is typical for operator-splitting methods applied to constrained dual formulations.



(a) Full outer-iteration range.

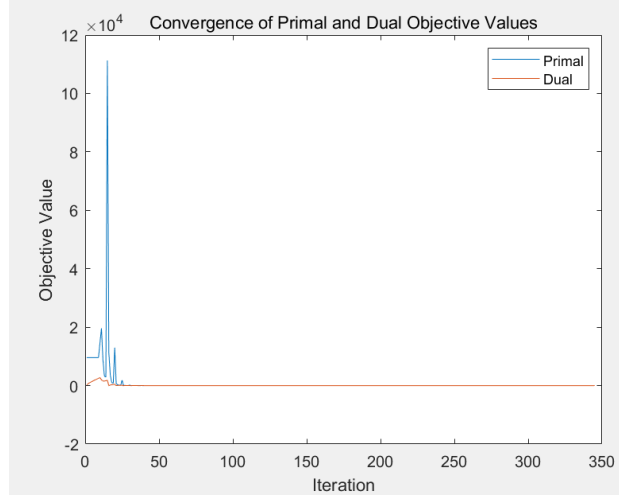


(b) After 50 outer iterations.

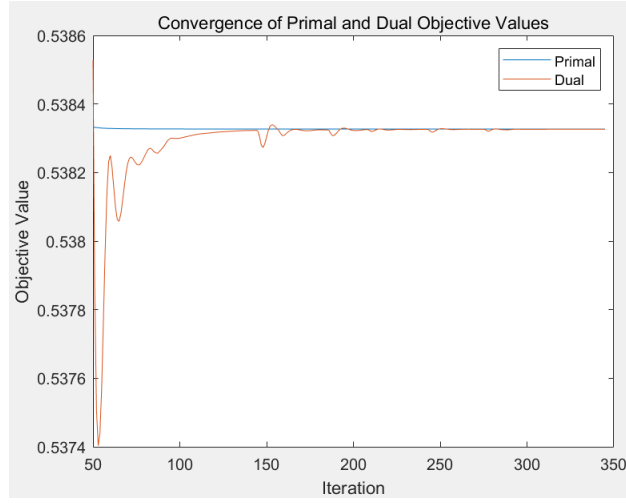


(c) After 600 outer iterations.

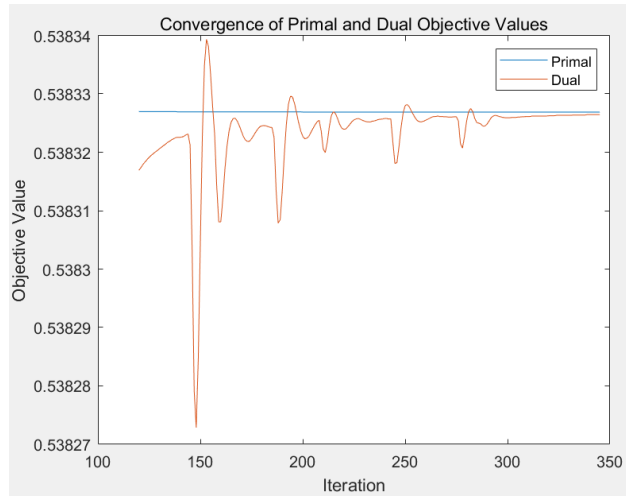
Figure 3: Convergence histories of the dual ALM method.



(a) Full iteration range.



(b) After 50 iterations.



(c) After 120 iterations.

Figure 4: Convergence histories of the dual ADMM method.

8 Problem 3(h): Linearized ADMM for the Primal Problem

8.1 Primal splitting and linearized update

To apply ADMM directly to the primal model, introduce a split variable $y = x$:

$$\min_{x,y} \frac{1}{2} \|Ax - b\|_F^2 + \mu \|y\|_{1,2} \quad \text{s.t.} \quad x - y = 0. \quad (24)$$

Using the scaled dual variable U , the augmented Lagrangian is

$$\mathcal{L}_\beta(x, y, U) = \frac{1}{2} \|Ax - b\|_F^2 + \mu \|y\|_{1,2} + \frac{\beta}{2} \|x - y + U\|_F^2 - \frac{\beta}{2} \|U\|_F^2.$$

If the x -subproblem is solved exactly, one must invert $A^\top A + \beta I$. To avoid this cost, a linearized ADMM step is used: linearize the smooth term at the current iterate and add a proximal term. More precisely,

$$x^{k+1} = \arg \min_x \left\{ \langle \nabla g(x^k), x - x^k \rangle + \frac{1}{2\tau} \|x - x^k\|_F^2 + \frac{\beta}{2} \|x - y^k + U^k\|_F^2 \right\}.$$

Since $\nabla g(x^k) = A^\top (Ax^k - b)$, the optimality condition gives the explicit update

$$x^{k+1} = \frac{x^k - \tau A^\top (Ax^k - b) + \tau \beta (y^k - U^k)}{1 + \tau \beta}. \quad (25)$$

The y -subproblem is still a proximal mapping:

$$y^{k+1} = \text{prox}_{(\mu/\beta)\|\cdot\|_{1,2}}(x^{k+1} + U^k), \quad (26)$$

which is evaluated by the rowwise block shrinkage formula (11). Finally,

$$U^{k+1} = U^k + x^{k+1} - y^{k+1}. \quad (27)$$

8.2 Discussion

The implementation uses a fixed step parameter $\tau = 0.1$ and solves the problem directly with the target value of μ , without continuation. The method remains accurate, but it is slower than dual ADMM and dual ALM. Linearization avoids solving a more expensive linear system at every step, but it also makes each iteration less accurate. As a result, the total iteration count increases to 2929, and the running time is higher than for the dual-splitting methods.

9 Conclusion

The numerical study leads to three main conclusions.

First, all tested methods are numerically consistent: they recover nearly the same objective value and nearly the same exact solution on the standard benchmark. Second, methods that exploit the composite structure of the group LASSO model—especially proximal-gradient and operator-splitting methods—are much more efficient than plain subgradient-type methods. Third, among the methods tested here, the dual ADMM solver provides the best overall performance on this benchmark, followed by dual ALM and the proximal-gradient family.

The continuation strategy in μ also plays an important practical role. By solving a sequence of easier problems first, it provides warm starts that substantially accelerate the final solve for the target regularization parameter.